

Part A - Function fundamentals

1. Write functions `greet()`, `show_course_name()` and `print_separator()`. Call each more than once.
2. Write `greet_person(name)` and `introduce(name, city)`.
3. Write `add(a, b)`, `subtract(a, b)`, `multiply(a, b)` and `divide(a, b)`. Each must return a value.
4. Demonstrate parameter vs argument in comments using one of your functions.
5. Create `calculate_area(width, height)` and use its returned value in another calculation.

Part B - Return values

1. Write `is_even(number)` returning `True/False`.
2. Write `get_larger(a, b)` returning the larger value without `max()`.
3. Write `classify_score(score)` returning `PASS` or `FAIL`.
4. Write `full_name(first_name, last_name)` returning a formatted string.
5. Write `calculate_discount(price, percent)` returning the discounted price.
6. Show with a small example why `print(result)` inside a function is not the same as `return result`.

Part C - Defaults and keyword arguments

1. Create `greet(name, greeting='Hello')`. Test positional and keyword arguments.
2. Create `calculate_price(price, quantity=1, discount=0)`. Return the final total.
3. Create `create_profile(name, city='Unknown', active=True)` returning a dictionary.
4. Call one function using keyword arguments in a different order from the parameter definition.
5. Write one invalid default-parameter ordering as a comment and explain why it is invalid.

Part D - Functions and collections

1. Write `calculate_total(numbers)` manually using a loop.
2. Write `count_even(numbers)`.
3. Write `get_long_words(words, minimum_length)` returning a new list.
4. Write `find_student(students, name)` where `students` is a list of dictionaries. Return the matching dictionary or `None`.
5. Write `average_score(students)` for a list of dictionaries containing score values.
6. Write `get_active_users(users)` returning only dictionaries where `active` is `True`.

Part E - Decomposition

1. Build a temperature report using separate functions for Celsius-to-Fahrenheit conversion, classification ('cold/warm/hot') and formatting.
2. Build a small order calculation using separate functions for subtotal, discount and final total.
3. Refactor one earlier exercise that contains repeated code into at least three functions.
4. Write a main-like section at the bottom of the file that calls your functions in a clear sequence.

Part G - Stretch challenges

1. Write a function that returns both minimum and maximum from a list without `min()/max()`. Return two values.
2. Write a function that checks whether a word is a palindrome.
3. Write a function that counts character frequencies and returns a dictionary
4. Write a function that receives a list of numbers and returns a new dictionary with keys `positive`, `negative` and `zero` containing counts.
5. Add light type hints and a short docstring to at least five functions.